```
module TP2 {

   // Définition des messages TFTP
   type integer UInt16 (0..65535);
   type charstring NullTerminatedString;

   type record TftpRequestPacket {
      UInt16         Opcode,
      NullTerminatedString   Filename,
      NullTerminatedString   Mode
   }

   type record TftpDataPacket {
      UInt16         Opcode,
      UInt16         BlockId,
      octetstring      Data
   }

   type record TftpAckPacket {
      UInt16         Opcode,
      UInt16         BlockId
   }

   type record TftpErrorPacket {
      UInt16         Opcode,
      UInt16         ErrorCode,
      NullTerminatedString   ErrMsg
   }

   type record TftpOtherPacket {
      UInt16         Opcode,
      octetstring      Data
   }

   type union TftpPacket {
      TftpRequestPacket   req,
      TftpDataPacket      data,
      TftpAckPacket      ack,
      TftpErrorPacket      err,
      TftpOtherPacket      other
   }

   // Définition des constantes Opcode
   const UInt16 TftpOpcodeRRQ      := 1;
   const UInt16 TftpOpcodeWRQ      := 2;
   const UInt16 TftpOpcodeDATA     := 3;
   const UInt16 TftpOpcodeACK      := 4;
   const UInt16 TftpOpcodeERROR    := 5;

   // Définition des constantes Errcode
   const UInt16 TftpErrorCodeUndefined          := 0;
   const UInt16 TftpErrorCodeFileNotFound       := 1;
   const UInt16 TftpErrorCodeAccessViolation    := 2;
   const UInt16 TftpErrorCodeDiskFull           := 3;
   const UInt16 TftpErrorCodeIllegalOperation   := 4;
   const UInt16 TftpErrorCodeUnknownTransferId  := 5;
   const UInt16 TftpErrorCodeFileAlreadyExists  := 6;
   const UInt16 TftpErrorCodeNoSuchUser         := 7;

   // Définition du port de communication TFTP
   type port TftpPort message {
      inout TftpPacket,
            TftpRequestPacket,
            TftpDataPacket,
            TftpAckPacket,
            TftpErrorPacket,
            TftpOtherPacket
   }

   // Définition des composants
   type component TFTpServer {
      port TftpPort ServerPort;
```

```
73      }
74      type component TFTpClient {
75         port TftpPort ClientPort;
76      }
77
78      template TftpPacket MyPacketANY := ?;
79
80      // Templates du testcase TC_1
81      template TftpRequestPacket MyPacketWRQ :=
82      {
83         Opcode := 2,
84         Filename := "toto",
85         Mode := "netascii"
86      }
87      template TftpAckPacket MyPacketACK :=
88      {
89         Opcode := 4,
90         BlockId := ?
91      }
92      template TftpErrorPacket MyPacketERR :=
93      {
94         Opcode := 5,
95         ErrorCode := ?,
96         ErrMsg := ?
97      }
98
99      // Templates du testcase TC_2
100     template TftpRequestPacket MyPacketRRQ :=
101     {
102        Opcode := 1,
103        Filename := "exemple.txt",
104        Mode := "netascii"
105     }
106
107     template TftpDataPacket MyPacketDATA :=
108     {
109        Opcode := 3,
110        BlockId := ?,
111        Data := ?
112     }
113
114
115     // testcase TC_1.1 : test d'ecriture d'un fichier sur le serveur
116     testcase TC_1() runs on TFTpClient system TFTpServer
117     {
118        map(self:ClientPort, system:ServerPort);
119        ClientPort.send(MyPacketWRQ);
120        timer t1 := 5.0;
121        t1.start;
122        alt {
123           [] ClientPort.receive(MyPacketACK){
124              setverdict(pass);
125              log("TC1 : Connexion etablie avec le serveur");
126           }
127           [] t1.timeout {
128              setverdict(fail);
129              log("TC1 : Timeout expire, connexion non etablie");
130
131           };
132           [] ClientPort.receive(MyPacketANY){
133              setverdict(fail);
134              log("TC1 : Le client recoit un paquet non attendu");
135           }
136        }
137        unmap(self:ClientPort, system:ServerPort);
138     }
139
140     // testcase TC_1.2 : test de lecture d'un fichier sur le serveur
141     testcase TC_2() runs on TFTpClient system TFTpServer
142     {
143        map(self:ClientPort, system:ServerPort);
144        ClientPort.send(MyPacketRRQ);
```

```
145         timer t1 := 5.0;
146         t1.start;
147         alt {
148            [] ClientPort.receive(MyPacketDATA){
149               setverdict(pass);
150               log("TC2 : Connexion etablie avec le serveur");
151            }
152            [] t1.timeout {
153               setverdict(fail);
154               log("TC2 : Timeout expire, connexion non etablie");
155
156            };
157            [] ClientPort.receive(MyPacketANY){
158               setverdict(fail);
159               log("TC2 : Le client recoit un paquet non attendu");
160            }
161         }
162         unmap(self:ClientPort, system:ServerPort);
163      }
164
165      // Décommenter selon le Test Case à exécuter
166      control {
167         //execute (TC_1());
168         //execute (TC_2());
169      }
170   }
```